RESEARCH-ARTICLE

Anytime Algorithms for Approximate Functional Dependencies

SANJIVNI RANA, The University of Texas at Arlington, Arlington, TX, United States

JUNYA OGAWA, The University of Texas at Arlington, Arlington, TX, United States

SURAJ SHETIYA, Indian Institute of Technology Bombay, Mumbai, MH, India

SENJUTI BASU ROY, New Jersey Institute of Technology, Newark, NJ, United States

GAUTAM DAS, The University of Texas at Arlington, Arlington, TX, United States

Published: 03 August 2025

KDD '25: The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining
August 3 - 7, 2025
Toronto ON, Canada

Conference Sponsors:
SIGMOD
SIGKDD

KDD '25: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (August 2025)
https://doi.org/10.1145/3711896.3736844
ISBN: 9798400714542

Anytime Algorithms for Approximate Functional Dependencies

Sanjivni Rana
sxr0277@mavs.uta.edu
University of Texas at Arlington
Arlington, TX, USA

Junya Ogawa
jxo3564@mavs.uta.edu
University of Texas at Arlington
Arlington, TX, USA

Suraj Shetiya
surajs@cse.iitb.ac.in
Indian Institute of Technology, Bombay
Mumbai, Maharastra, India

Senjuti Basu Roy
senjutib@njit.edu
New Jersey Institute of Technology
Newark, NJ, USA

Gautam Das
gdas@uta.edu
University of Texas at Arlington
Computer Science and Engineering
Arlington, TX, USA

Abstract

We propose a computational framework for identifying approximate functional dependencies (AFDs) in a relation, leveraging the frequency distribution information of individual attributes. This framework operates without requiring access to the full database, processing records one at a time as necessary. Our approach generalizes existing measures for quantifying errors in perfect dependencies and formalizes two primary problems: finding top-k AFDs and identifying all AFDs within a specified error threshold, ϵ. Our proposed framework provides anytime solutions, meaning it returns results after processing each record. A key innovation of our work lies in effectively estimating error bounds of the candidate AFDs, which allows to produce anytime solutions. We present an exact algorithm that delivers precise solutions when possible. We also develop an algorithm that always returns a solution albeit with some imprecision in the output. We demonstrate the applicability of these algorithms under various data organization strategies, such as indexing by key or key-like attributes. Our experimental results, based on both real-world and synthetic datasets, validate the effectiveness of our approach and show that it outperforms state-of-the-art solutions.

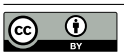
CCS Concepts

• **Information systems** → **Data cleaning**; • **Theory of computation** → *Database constraints theory*; *Linear programming*.

Keywords

Approximate Functional Dependency, Data cleaning, Anytime Algorithm

ACM Reference Format:
Sanjivni Rana, Junya Ogawa, Suraj Shetiya, Senjuti Basu Roy, and Gautam Das. 2025. Anytime Algorithms for Approximate Functional Dependencies. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3736844



KDD '25, Toronto, ON,Canada

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3736844

KDD Availability Link:
The source code of this paper has been made publicly available at https://github.com/SanjivniRana/Anytime-AFDs.

1 Introduction

Functional dependencies (FDs) state that some attribute(s) in a relational instance functionally determine the value of another attribute. They are useful for discovery of hidden patterns, data cleaning [28–30, 33], or predictive modeling [26]. However, in real-world databases, functional dependencies are often approximate. The notion of *approximate* functional dependencies (AFDs) is defined by Kivinen et al. [21] to overcome this and has found significant research interests [5, 22, 23]. Given a set of attributes X (on the left hand side) and the "*right hand side*" attribute Z, AFD measures the *error* of the given dependency ($X \rightarrow Z$). Kivinen et al. [21] propose three different error measures for error in AFDs - g_1, g_2 and g_3.

We approach the problem of identifying AFDs in scenarios, where some limited distributional information about the data is available, but accessing actual data records incurs cost. This setting is common in data marketplaces[7, 13], platforms like AWS Data Exchange, Datarade, and Data & Sons, etc. In these marketplaces, individual attribute distributions are provided to help potential buyers assess the usefulness of datasets, but accessing more records from the database incurs incremental costs. Our work is also relevant for analytics over deep web databases [3, 11, 46], where vast repositories of data are accessible only through restricted web interfaces. Examples include platforms like Kayak, Expedia, Hotels.com, or Amazon, which display attribute-level metadata—such as distributions of flight prices, departure times, or number of stops—but limit direct access to data by showing only the top-k results per query. Moreover, these interfaces often impose constraints on the number of allowable queries, further restricting full data access. In such constrained environments, where accessing full datasets is impractical or expensive, our work provides techniques to discover AFDs without requiring extensive data access.

We present a computational framework under the following settings: (a) frequency distributions of the individual attributes are known apriori to the framework; (b) access to the database is enabled through a getNext() [43] interface that returns the next data record, or the next small batch of data records when invoked, but how the actual data is stored and organized is unknown to the

framework; and (c) more getNext() calls incur more cost and hence it is better to minimize those calls.

Problems & Contributions. Anytime AFDs. We initiate the problem of designing *anytime algorithms* for computing AFDs in a relation considering three different error measures [21], where the actual records are read one by one, or in small batches. Anytime algorithms [2, 47] are designed to progressively improve the quality of their results as more computation time becomes available, while still being capable of returning a valid (though possibly suboptimal) solution if interrupted at any stage. Our proposed algorithms fall within a subclass of anytime algorithms that begin producing outputs after reading only a small portion of the data. A formal definition is provided in Section 2. We study two problem variants.

- **[top-k length ℓ-AFDs.]** At any point during reading the records, retrieve the top-k AFDs (i.e., the k AFDs with the smallest errors), each with exactly ℓ attributes at the left-hand side, where k and ℓ are application defined tunable input parameters.
- **[Minimal ϵ-AFDs.]** At any point during reading the records, determine all minimal AFDs (an AFD is minimal when no proper subset of it has error $\leq \epsilon$) where ϵ is an application defined threshold.

Our proposed overarching solution framework designs a powerset lattice of all but the right hand size attribute Z, where each attribute set X forms a candidate AFD, representing $X \rightarrow Z$. To identify Minimal ϵ-AFDs, the search starts with the AFD candidates that are at the bottom of the lattice, gradually makes its way up level by level, and makes a decision whether to keep the candidate AFD or prune it. For top-k length-ℓ AFDs, the algorithm identifies all size-ℓ AFD's and makes decisions whether to prune a candidate AFD.

Sub-problems and designed solutions. Recall that our goal is to design an anytime solution framework where we wish to return the top AFDs for the entire relation even when only a subset of records are read. This requires determining which AFDs are likely to be the solution and eliminating those that cannot be. This decision-making process is facilitated by calculating *guaranteed upper and lower bounds for the error for the candidate AFDs, which is a central contribution of our work.*

We define two core sub-problems: Given n out of N records have been read, the *Maximum Error Estimate*, which calculates the upper bound on the error of an AFD for a given error measure, and the *Minimum Error Estimate*, which calculates the lower bound. We explore various optimization techniques for both problems across three error measures (g_1, g_2, and g_3), using methods, like, mixed integer linear programming (MILP), linear programming, greedy algorithms, and quadratic programming. Additionally, we provide additive approximation guarantees for the g_3 error measure and discuss strategies to further improve scalability.

We then present algorithms for our two main problems that make use of the upper and lower bounds. We propose an exact algorithm ESCA for the top-k length ℓ-AFDs problem that generates a prefix k' (with $k' \leq k$) of AFDs, which grows as more records are processed. Additionally, we present an approximate solution based on estimations that consistently identifies k AFDs, though they may not always be the true top-k. This approach follows the *uninformed prior principle*, assuming uniform error in unread records, but ensuring the final frequency distribution matches individual attribute distributions.

We propose an exact algorithm EXEA for the Minimal ϵ-AFDs problem that explores the powerset lattice bottom-up, making decisions on whether to keep or prune candidate AFDs at each level. Unlike classical minimal AFD algorithms that assume full knowledge of the relation, we establish monotonic relationships between the upper and lower bounds of candidate AFDs and their ancestors. This enables our algorithm to guarantee that at any point, a proper subset of ϵ-AFDs is returned, with the subset growing as more records are read. We also present a solution based on estimations, which may not always identify the true minimal ϵ-AFDs but works well in practice and are highly efficient, relying on the principle of *uninformed prior.*

Experimental Evaluations. We conduct extensive experimental evaluations using 3 real world and multiple synthetic datasets. We note that there does not exist any related work [22, 23] that study anytime AFDs. Nevertheless, we adapt the state-of-the-art (SOTA) solution Pyro [23] which is not guaranteed to produce exact solutions, and compare against ours, wherever applicable. We also adapt a classical solution TANE [18] to produce exact solutions. We implement a machine learning based solution based on iterative proportional fitting for comparison [40]. Our evaluations reveal three key findings: (a) We illustrate the applicability of each algorithm across different data organization strategies, comparing scenarios where data is indexed by key or key-like attributes versus cases where it is not. Our findings indicate that our proposed solutions that estimate error based on both read and unread data outperform the others convincingly. (b) Our proposed algorithms are highly effective - after reading about 40% of the data, they achieve high precision and NDCG score. (c) The efficiency opportunities discussed work well in practice and the bound computation becomes scalable.

2 Data Model & Problem Definition

We present our data model in Section 2.1, and formally define the problems in Section 2.2.

Running Example. Table 1 shows our running example over a toy dataset of 6 records. `Tuple ID` is the unique primary key in this table. The other attributes are `First Name`, `Gender`, `ZIP`, and `Town`.

Tuple ID	First Name	Gender	ZIP	Town
t1	Alex	f	12345	Linden
t2	John	m	12346	Linden
t3	Jane	f	12355	Hanover
t4	John	m	12346	Linden
t5	Alex	m	12345	Linden
t6	Tia	f	12345	Linden

Table 1: A toy database

2.1 Data model

Database. Consider a database $\mathcal{D}$ with N tuples and d categorical attributes ($\mathcal{A} = \{ A_1, A_2, \cdots, A_d \}$). We refer to the i^{th} tuple as t_i and value for the j^{th} attribute for tuple t_i as $t_i[j]$.

getNext(). The access to $\mathcal{D}$ is performed by making getNext() calls that returns the next record from the database. How data is stored in $\mathcal{D}$ is unknown, and getNext() calls are expensive.

Domain of an attribute. m_A is the size of the domain of attribute A which is the number of unique values A can take. Domain size of $m_{\text{First Name}} = 4$.

Known Frequency/Marginal distributions. The frequency/ marginal distribution of each $A \in \mathcal{A}$ is known to the framework. Let m be the domain size of an attribute A, where f_ℓ represents the frequency of the ℓ-th value of the domain, such that $\Sigma_{\ell=1}^{m} f_\ell = N$. Using the running example, m of Gender is 2, $f_{male} = 3$, $f_{female} = 3$, $f_{male} + f_{female} = 6$.

Attribute set X, attribute Z. X is a set of attributes $X \subset A$, but Z is a single special attribute, $Z \subset A$, $X \bigcap Z = \{\}$, $|Z| = 1$. $\mathcal{M}_{\mathcal{A}}^{X,Z}$ represents the set of marginal distributions of attribute X and Z. Using the running example, marginal distribution of *(First Name)* is $f_{(Alex)} = 2$, $f_{(John)} = 2$, $f_{(Jane)} = 1$, $f_{(Tia)} = 1$. Marginal distribution of `Gender` is $f_{male} = 3$, $f_{female} = 3$. However, the joint distribution of these two attributes are not known until the full database is read.

Functional Dependency: Given a database $\mathcal{D}$, set of attributes $X \subset \mathcal{A}$ and $Z \in \mathcal{A}$, a functional dependency from X to Z is written as $X \rightarrow Z$. The dependency $X \rightarrow Z$ (read as X determines Z) holds, if for all pair of tuples that agree on X also agree on Z, i.e., when $t_i[X] = t_j[X]$, then $t_i[Z] = t_j[Z]$. Using Example 1, the only FDs that exist are Tuple Id $\rightarrow$ any of the 4 other attributes. Clearly, FDs involving Tuple Id are not useful.

Violation: For a given database $\mathcal{D}$, set of attributes $X \subset \mathcal{A}$ and $Z \in \mathcal{A}$, two tuples t_i and t_j are considered a violating pair if $t_i[X] = t_j[X]$ but $t_i[Z] \neq t_j[Z]$. A tuple t_i is called violating if it is a part of some violating pair. Using Table 1, (t_1, t_5) is a violating pair: $t_1[FirstName] = t_5[FirstName]$ but $t_1[Gender] \neq t_5[Gender]$.

Approximate Functional Dependency: Given a dataset $\mathcal{D}$, a set of attributes $X \subset \mathbb{A}$ and the ("*right-hand side*") attribute Z, AFD determines the *error* for the given dependency ($X \rightarrow Z$).

In this work, we look at three different error measures defined in Kivinen et. al. [21]. The definitions of the three errors G_1, G_2 and G_3 and their corresponding scaled measures (g_1, g_2 and g_3) are defined below

- G_1 error: The number of violating pairs in $\mathcal{D}$.
$$G_1(X \rightarrow Z, \mathcal{D}) = |\{(t_i, t_j) \mid t_i[X] = t_j[X], t_i[Z] \neq t_j[Z]\}|$$
$$g_1(X \rightarrow Z, \mathcal{D}) = G_1(X \rightarrow Z, \mathcal{D})/N^2$$
- G_2 error: the number of violating tuples in $\mathcal{D}$.
$$G_2(X \rightarrow Z, \mathcal{D}) = |\{t_i \mid \exists t_j \in \mathcal{D}, t_i[X] = t_j[X], t_i[Z] \neq t_j[Z]\}|$$
$$g_2(X \rightarrow Z, \mathcal{D}) = G_2(X \rightarrow Z, \mathcal{D})/N$$
- G_3 error: The number of tuples that have to be deleted to satisfy the functional dependency in $\mathcal{D}$.
$$G_3(X \rightarrow Z, \mathcal{D}) = N - max\{|s| \mid s \in \mathcal{D},\ s \models X \rightarrow Z\}$$
$$g_3(X \rightarrow Z, \mathcal{D}) = G_3(X \rightarrow Z, \mathcal{D})/N$$

The $\models$ operator refers to all s for which the functional dependency $X \rightarrow Z$ holds. Note that the scaled errors g_1, g_2 and, g_3 are in the range of $[0, 1]$. Additionally, g_1 error has also been formulated [23] by normalizing with $N(N-1)$. While we use the notion from Kivinen et al. [21], our proposed approaches work with both measures.

The AFD First name$\rightarrow$ Gender has one violating pair (Alex,m) and (Alex, f). As g_1 error counts the number of such pairs and there is only 1 violating pair the g_1 error is $2/36$. g_2 error counts the number of records which are in violation and hence is $2/6$. For the FD to hold, deleting either (Alex,m) or (Alex, f) would suffice. Hence, g_3 error is $1/6$.

2.2 Problem Definitions

We define two broad classes of problems.

[top-k length ℓ-AFDs.] Given n out of N records of the $\mathcal{D}$ are read, a tunable parameter ℓ, marginal distributions $\mathcal{M}_{\mathcal{A}}$ of $\mathcal{A}$, an error measure (g_1, g_2 or g_3), an integer k, retrieve as a ranked list the top-k AFDs having the least errors (along with their respective error scores, if possible) from among all AFDs having exactly ℓ attributes in the left hand side (LHS).

Practical choice of ℓ: Note that as the attributes on the LHS grows, the error monotonically decreases. However, having more attributes on the LHS does not necessarily provide meaningful insights on data quality and correlation. For instance, when all but one attribute is on the LHS, a trivial full functional dependency exist. That is why we introduce an input parameter ℓ, which constrains the maximum number of attributes allowed on the LHS. This parameter allows us to explore meaningful and compact dependencies while avoiding trivial cases.

[Minimal ϵ-AFDs.] Given n out of N records of the $\mathcal{D}$ are read, marginal distributions $\mathcal{M}_{\mathcal{A}}$ of $\mathcal{A}$, an error measure (g_1, g_2 or g_3), and an error threshold ϵ, retrieve all minimal AFDs which have an error score of at most ϵ along with their respective error scores. An AFD $X \rightarrow Z$ is a minimal ϵ-AFD if no proper subset of X is also an ϵ-AFD to determine Z.

2.3 Definition of Anytime Algorithms for AFDs

Before proposing our solutions to the above problems, we would like to define the notion of *Anytime Algorithms* in the context of computing AFDs. Our definition hinges on a few observations.

For each of the proposed problems, the output is a set of AFDs. Let AFD($\mathcal{D}$) represent the set of output AFDs that are identified after reading the entire database $\mathcal{D}$. We refer to this as the optimal output. Consider an algorithm that has only read part of the database, say $\mathcal{D}'$, yet is able to output a set of AFDs, say AFD($\mathcal{D}'$) as the solution for the full database. Such an algorithm is called an Anytime Algorithm.

For example, consider the scenario of reading the first three rows of the sample dataset from Table 1 and g_3 error be the error measure and $\ell = 1$. Let the set of AFDs being considered for the top-k be First Name $\rightarrow$ Gender, Zip $\rightarrow$ Gender and Town$\rightarrow$ Gender. Based on the first three rows and known $\mathcal{M}_{\mathcal{A}}$, is it possible to get the top-1 AFD among the ones being considered? If not, which one is the *most likely* top-1 AFD among the ones being considered? These are the type of decisions an anytime algorithm has to consider.

Furthermore, an anytime algorithm is *well behaved* if the "similarity" of AFD($\mathcal{D}'$) to AFD($\mathcal{D}$) progressively improves as it reads more data. Similarity can be measured by several standard measures such as precision, recall, NDCG measure [45], etc.

In this paper we design several anytime algorithms for solving our proposed AFD problems. Our exact approaches designed in Section 4.1 are well-behaved anytime algorithms whenever the precision is always 1, while the recall monotonically increases as more data are read. We also propose "*approximate*" approaches in Section 4.2 where the precision and recall do not always exhibit monotone behavior, but the recall is often better than the Exact approaches (See Experiments Section 5).

3 Solution Framework: Bound Computation

In this Section, we discuss the proposed solution framework and motivate the need for computing lower and upper bounds of the errors of AFDs for producing anytime answers. We conclude this section, by presenting our proposed algorithms for obtaining lower and upper bounds of g_3 error, and add a pointer to the Appendix for our algorithms for the other error measures.

3.1 Lattice Based Solution

Our techniques rely on a lattice structure, which has been extensively used in Frequent Item-set Mining literature [25]. Our overarching solution framework designs a powerset lattice of all but the right-hand size attribute Z, where each attribute set X directly forms a candidate AFD, representing $X \rightarrow Z$. The first level of the lattice consists of single LHS AFDs (of the form $X \rightarrow Z$). The second level of the lattice consists of AFDs which have two attributes in the LHS. Each AFD $A_iA_j \rightarrow Z$ in the second level is connected to the AFDs $A_i \rightarrow Z$ and $A_j \rightarrow Z$ with two directed edges from the two level 1 nodes to the level 2 nodes. We refer to $A_i \rightarrow Z$ and $A_j \rightarrow Z$ as descendants of $A_iA_j \rightarrow Z$. The vice versa relation is termed as the ascendant relation. Based on our description, this lattice structure with $d-1$ level forms a Directed Acyclic Graph.

At a high level, the algorithms developed for both problems (described in Section 4) traverse the lattice and make decisions about whether to retain or prune candidate AFDs based on their error. However, the key challenge in enabling anytime decision-making is as follows: *how to compute the error of a candidate AFD, considering the data that has been read and the data that is yet to be read? This requires estimating how the unread data might influence the error of a candidate AFD.* For that, the framework utilizes error bounds, including both lower (minimum) and upper (maximum) bounds. These bounds are determined based on the data read thus far and the potential impact of the unread data, taking into account the global frequency distribution of the individual attributes.

3.2 Compute Bounds for Anytime Answer

Since the AFD algorithms (Section 4) generate solutions even when the entire data is not read, they must rely on the *lower and upper bounds of errors for candidate AFDs* and based on that make a decision. We formally define error bounds next.

3.2.1 Lower and Upper Bounds.

Minimum/Maximum error estimate satisfying marginal distribution constraints: Given n tuples of the database $\mathcal{D}$ are read, for a given AFD $X \rightarrow Z$, and an error measure (g_1, g_2 or g_3), produce minimum or lower (respectively maximum or upper) bound of error, which is the smallest (respectively largest) possible error that $X \rightarrow Z$ can have, while satisfying the marginal distribution constraints $\mathcal{M}_{\mathcal{A}}^{X,Z}$.

Knowing these upper and lower bounds at any time offer pruning opportunities. For instance, given two AFDs $X \rightarrow Z$ and $Y \rightarrow Z$, if the upper bound of error of $X \rightarrow Z$ is not larger than the lower bound of error of $Y \rightarrow Z$, then $X \rightarrow Z$ could be selected as the winner between these two.

As an example, assume the first three records in the Table 1 have been read. For the AFD First Name→ Gender, the current g_3 error is 0, however, based on the marginal distributions of `First Name` and `Gender` - one possible assignment of the remaining three records satisfying the marginal constraints could be the following - John → m, Alex → f, Tia → m. This is the best possible arrangement and gives a lower bound of g_3 error of 0. However, another possible assignment of the remaining three records satisfying the marginal constraints could be - John → m, the Alex → m, Tia → f, giving rise to upper bound of g_3 error to be 1.

3.2.2 Bounds Computation Problem.

Bound computation problems take "*joint distribution frequency matrices*" as inputs, one for every AFD $X \rightarrow Z$ under consideration based on the n records it has read so far. A joint distribution frequency matrix of $X \rightarrow Z$ has m_Z columns and m_X rows, where m_Z (similarly m_X) is the domain size of attribute Z (similarly X).

Let a cell in the r^{th} row and c^{th} column of the matrix be denoted as $S[r][c]$. Given n records are read so far, $S[r][c]$ stores number of records out of n, where X has r^{th} and Z has c^{th} value from their respective domains. The designed lower and upper bound solutions need to consider marginal distributions $\mathcal{M}_{\mathcal{A}}^{X,Z}$ as well and *estimate the smallest and largest of the given error measure*, since $N-n$ records are yet to be read. Based on the error measures, the abstract problem is as follows: *As the $N-n$ records are yet to be read, among all the unseen joint distribution frequency matrices which satisfy the marginal information $\mathcal{M}_{\mathcal{A}}^{X,Z}$, find that which minimizes (maximizes respectively) the given error measure thus finding the lower (upper respectively) bound.*

The problem is computationally intractable under certain settings (refer to Appendix A).

We develop a suite of techniques leveraging Mixed Integer Linear Programming (MILP), Linear Programming (LP) relaxation, and Greedy strategies to derive the bounds. Our methods, including Greedy algorithms and MILP formulations implemented with solvers such as Gurobi [17], are designed to be both fast and practical. In contrast, approaches based on Dynamic Programming (DP) [4] tend to incur significantly higher computational costs. We defer the development of efficient and approximate DP-based solutions to future work (see Section 7).

3.3 Bound Computation Algorithms

For limitation of space, from now on, we primarily focus on g_3 error and focus on minimization as long as its counterpart (maximization) could use the same technique.

Mixed Integer linear programming (MILP) [44] can be used as a tool to formalize and solve the minimization/maximization problems. These problems are different from Integer Linear Programs(ILP) [41] in a sense that MILP require only some of the variables to take integer values, whereas ILP problems require all variables to be integers.

As part of the input, n rows have been read into a joint distribution frequency matrix. The remaining $N-n$ rows which will be represented by an unseen joint distribution frequency matrix U needs to satisfy the marginal information $\mathcal{M}_{\mathcal{A}}^{X,Z}$. The goal is to produce the unseen joint distribution frequency matrix.

For AFD $X \rightarrow Z$, U is a matrix of size $m_X \times m_Z$. $U[r][c]$ is a variable that is defined on r-th row and the c-th column over U that takes non-negative integer values, $U[r][c] \in \mathbb{W}$.

$$
\begin{aligned}
&maximize \ \sum_{r=0}^{m_X-1} max[r] \\
&s.t.\ \forall r \in \{0,..,m_X-1\}\ \forall c \in \{0,..,m_Z-1\} \quad U[r][c] \in \mathbb{W} \\
&\quad \forall r \in \{0,..,m_X-1\} \quad row[r] = \sum_{a=0}^{m_Z-1} S[r][a] + U[r][a] \\
&\quad \forall c \in \{0,..m_Z-1\} \quad col[c] = \sum_{a=0}^{m_X-1} S[a][c] + U[a][c] \\
&\quad \forall r \in \{0,..,m_X-1\} \quad \forall\, c \in \{0,..,m_Z-1\} \\
&\quad\quad max[r] \geq S[r][c] + U[r][c] \\
&\quad\quad max[r] \leq S[r][c] + U[r][c] + (1 - \mathcal{B}[r][c]) * N \\
&\quad\quad \mathcal{B}[r][c] \in \{0,1\} \\
&\quad \forall\, r \in \{0,\cdots,n-1\} \quad \sum_{c=0}^{m_Z-1} \mathcal{B}[r][c] = 1
\end{aligned}
$$

Figure 1: Mixed ILP for g_3 error

As the sum of the matrices $U + S$ must satisfy the marginal information, we add additional constraints to restrict the values in U. The constraints that correspond to the marginal information on the attribute X are defined below.

$$\forall r \in \{0,..,m_X-1\} \qquad \mathcal{M}^X_{\mathcal{A}}[r] = \sum_{a=0}^{m_Z-1} S[r][a] + U[r][a]$$

Similar constraints for Z can be defined.

We introduce a variable for each row of $U + S$ that counts of the records to be retained. In every row (which is a unique domain value) of $U + S$, the maximum value in the row represents the number of records that will be retained for that domain. Thus, we introduce m_X variables $\texttt{max}[0], \cdots, \texttt{max}[m_X\text{-}1]$ which counts these records in every row. Based on max, the optimization function can thus be expressed as,

$$minimize \ \ N - \sum_{r=0}^{m_X-1} max[r]$$

This is equivalent to

$$maximize \ \ \sum_{r=0}^{m_X-1} max[r]$$

We introduce constraints such that $max[r]$ is greater than all values in the row r of final joint distribution frequency matrix $S+U$.

$$\forall r \in \{0,..,m_X-1\}\ \forall c \in \{0,..,m_Z-1\} \ \ max[r] \geq S[r][c] + U[r][c]$$

We need additional constraints to restrict $\texttt{max}[r]$ to be equal to the maximum value in row r. To add this restriction, we introduce $m_X \times m_Z$ *boolean* variables $\mathcal{B}$ that indicate the location of the maximum value in each row r of $S + U$. As each row can only contain one maximum value, only one variable in each row can be *true* (1). The inequalities needed are listed below,

$$
\begin{aligned}
&\forall r \in \{0,\ldots,m_X-1\}\ \forall c \in \{0,..,m_Z-1\} \\
&\qquad max[r] \leq S[r][c] + U[r][c] + (1 - \mathcal{B}[r][c]) * N \\
&\qquad \forall c \in \{0,..,m_Z-1\} \ \ \mathcal{B}[r][c] \in \{0,1\} \\
&\forall r \in \{0,\ldots,m_X-1\} \sum_{c=0}^{m_Z-1} \mathcal{B}[r][c] = 1
\end{aligned}
$$

The final MILP is presented in Figure 1.

3.3.1 Accuracy-Runtime Trade Offs. The MILP in Figure 1 produces the exact result, but Mixed Integer Linear Programs are known to be computationally expensive.

In this section, we discuss two efficiency opportunities that are likely to expedite the original optimization problem defined in Section 3.3 at the cost of producing "looser" bounds. The first one relaxes the MILP to a Linear Program (LP) and the second one relaxes some marginal distribution constraints.

LP Relaxation: Note that the MILP for the lower bound of g_3 error in 3.3 consists of two matrices of Integer variables, $\mathcal{B}$ (*Boolean*) and U (non-negative integer). Hence, we relax the *Boolean* variables between 0 and 1. The non-negative integer variables are relaxed to take non-negative real values. Based on these modifications, the relaxed LP is written in Figure 10 in Appendix.

We prove that the LP relaxation for the upper bound of g_3 error has an additive error of at most $O(m_X)$.

Theorem 3.1. *The relaxed Linear Programming formulation finds the upper bound of g_3 error with an additive error of at most $O(m_X)$.*

The proof is moved to the Appendix B. Further details about optimizations is presented in Appendix C.2.1.

4 Anytime AFD Algorithms

In this section, we discuss two types of algorithms for the two main problem variants. These algorithms make use of the lattice structure (Section 3.1) and use the upper and lower bounds (Section 3.2).

For the top-k length ℓ-AFDs problem, we propose an exact algorithm that progressively generates a prefix k' ($k' \leq k$) of the top-k AFDs as more records are processed. We also offer an approximate solution that always identifies k AFDs, though not always the true top-k, based on uniform error assumptions for unread records.

For the Minimal ϵ-AFDs problem, we introduce an exact algorithm that explores the powerset lattice bottom-up, using upper and lower bounds to prune candidate AFDs. This approach ensures that a proper subset of ϵ-AFDs is returned as records are read. Additionally, we present an approximate algorithm, based on uninformed prior, which works well in practice though it may not always identify the true minimal ϵ-AFDs.

For simplicity, we use "top-k" for finding top-k AFDs with ℓ attributes on the left side, and "ϵ-AFDs" for the minimal ϵ-AFDs.

4.1 Exact Algorithms

The AFDs generated by the exact algorithms are always correct, meaning they belong to the actual top-k or ϵ-AFDs. However, since these algorithms make decisions anytime without having access to the entire dataset, the results they identify may only represent a ($k' \leq k$) subset of the full outcomes, depending on the amount of data they have read at any given point in time.

4.1.1 Top-k AFDs. The algorithm ESCA (Early Stop Criteria based Top-k AFD Algo) starts at level ℓ of the lattice considering all nodes at that level. Since $\binom{d-1}{\ell}$ is likely to be larger than k, it just has to find the top-k from there. Since only a subset of the data is read, ESCA needs to determine the correct top-k ranked order.

ESCA constructs a directed graph G with $\binom{d-1}{\ell}$ no of nodes (could be represented using Hasse diagram [12]), and the edges represent the partial order between every pair of AFDs. The nodes of the graph are the AFDs and a directed edge from vertex ($X \rightarrow Z$) to vertex ($Y \rightarrow Z$) indicates that $X \rightarrow Z$ has lower error than $Y \rightarrow Z$. Initially, when no data is read, there are no edges in the graph. Edges are added as more data is read, and the order between AFDs could be established. As a simple optimization, if any vertex (AFD) in the graph has more than k incoming edges, it is already dominated by k other AFDs and thus it is eliminated from G as it cannot be part of the solution. This ensures a precision of 1 as no false positives are added to the set of AFDs.

However, the challenge is the following: When only a part of the data is read, ESCA will not be able to quantify the exact error of

any node. Thus, it relies on the *upper (largest)* and *lower (smallest)* bounds of errors of AFDs for comparison and making decisions (formally defined and studied in Section 3.2). As an example, if the *upper bounds* of error of $X \rightarrow Z$ is smaller than the *lower* bound of error of $Y \rightarrow Z$, then $X \rightarrow Z$ is placed before $Y \rightarrow Z$.

To find the next-best AFD, the algorithm does the following: (a) finds if there exists an AFD whose lower bound of error is not larger than the upper bound of errors of all other AFDs. (b) If the answer is *yes*, then that AFD is returned, and it is eliminated from the current graph. These steps are repeated until Top-k AFDS are identified. (c) Steps (a) and (b) are repeated k' ($\leq k$) times to find top-k' AFDs in this fashion. If $k' < k$, the algorithm continues to issue more getNext(). Otherwise, the algorithm halts. The recall of the Exact anytime algorithm depends on the value of k'. If k' is near k, the recall is large. But an important thing to note is that as more data is read, k' can only increase. Hence, our Exact Anytime algorithm has a precision of 1 and a monotone recall property. As our algorithm exhibits an increasing monotonic nature for the similarity between the set of AFDs it produces and the final set of AFDs, our Exact algorithm is *well-behaved Anytime AFD Algorithm.*

4.1.2 ϵ-AFDs. For finding ϵ-AFDs, Algorithm EXEA (Early Stop Criteria based ϵ-AFD Algorithm) starts at the bottom of the lattice. First, it identifies all singleton AFDs and obtains their respective lower and upper bounds of errors. If the upper bound of error for an AFD is at most ϵ, then that AFD is identified as a minimal AFD and added to the set of results. From then on, all of its super sets are eliminated from further consideration, since none of these latter AFDs will be minimal. On the other extreme, if the lower bound of error of an AFD exceeds ϵ, that AFD is eliminated without further consideration. The process goes level-by-level until the whole lattice is explored. The pseudocode is in the Appendix (Algorithm **??**).

Theorem 4.1. *Given n records of the database are read, Algorithms ESCA and EXEA find AFDs such that these AFDS are a subset of OPT*, where OPT* is the optimal set of AFDs for the respective problem. Moreover, ESCA and EXEA are well-behaved Anytime Algorithms as the precision is always equal to 1 and recall is monotone, increasing.*

The proof is in the Appendix B.

4.2 Approximate Algorithms

At any point in time during the execution, approximate algorithms always return answers [2], (e.g., k answers for top-k AFDs even if only a very small part of the data is read), which may be imprecise, meaning that the identified AFDs may not always represent the true top-k AFDs or ϵ-AFDs.

4.2.1 Overall Idea. These algorithms are designed based on the idea of uninformed prior, which is defined below.

Uninformed prior: The distribution of unread records is uniform (based on the principle of indifference) as long as they satisfy marginal constraints.

Given n out of N records are read, the approximate algorithms intend to do the following: design an *efficient algorithm that outputs a small yet uniform sample of future distribution of joint distribution frequency matrices such that the marginal constraints are satisfied.* We note that this problem of uniformly sampling from the future distributions of joint-distribution frequency matrix under marginal constraints is connected with seemingly unrelated problems that has been long studied by the theoretical computer science and computational geometry community, such as volume estimation and *almost* uniform lattice sampling of convex polytopes [16, 20] under some weak assumptions.

Unfortunately, the unseen joint distribution frequency matrix in our case is very high-dimensional, hence these sampling techniques are not practical in our scenarios. We instead design multiple solutions that are highly efficient, work well in practice, and are not affected by the curse of dimensionality. Our approaches are based on a different notion of uninformed prior, where we assume that the error distribution is uniformly distributed. I.e., we leverage the error bounds of each candidate AFD and assume its error value is uniformly distributed within its respective (lb, ub) interval of error. We refer to Section C.3 in the Appendix for further details.

4.2.2 Top-k AFDs. The algorithms discussed in Section C.3 assigns an error to each AFD based on read and unread data. Now the approximate Top-k AFD algorithm traverses the lattice like its exact counterpart EXCA, but at every node in the lattice it identifies top-k as the one with the k smallest expected errors, or the most frequently occurred Top-k order (further details on both are discussed in Section C.3 in the Appendix).

4.2.3 ϵ-AFDs. The approximate algorithm for ϵ-AFDs traverses the lattice same as its exact counterpart EXEA and makes identical decision. However, like the approximate top-k AFD algorithm, it takes into consideration the assigned error of each candidate AFD from the Algorithms in Section C.3. Based on those error estimates, it decides to either output that candidate AFD (if the assigned error is $\leq \epsilon$) or prunes it (otherwise) at that step.

5 Experiments

Our experimental analyses are designed on two broadly defined goals - (i) *Quality.* demonstrates the applicability of each algorithm across different data organization strategies considering different error measures studied. Data is organized using mainly two strategies, i.e. indexing data by key attributes and indexing data by non-key like attributes. (ii) *Scalability.* measures running time of the proposed algorithms by varying pertinent parameters.

5.1 Experimental setup

Datasets: We use three real-world and synthetically generated datasets. A brief description of our datasets are as follows:

- *Student*: Student dataset [19] contains many attributes like gender, race, parental education etc which has often been used to analyze student's performance.
- *Letter*: Letter dataset [24] consists of data from black-and-white rectangular pixel displays for the 26 capital letters in the English alphabet.
- *DB Status*: DB Status [32] represents internal RCSB records to keep track of data processing and status of the entry.
- *Synthetic*: The data generation process begins by setting the domain size and distribution (uniform or normal) of the target attribute Z. Then, for each determining attribute X (on the LHS of a AFD), its domain size is specified. Each X is assigned a 1-D distribution based on the strength of its functional dependency with Z—categorized as Very Strong, Strong, Weak, or Very Weak.

Dataset	# Records	# Attributes
Student	1000	5
Letter	20,000	5
DB Status	10,000	7
Synthetic Dataset	100,000	10

Table 2: Statistics of datasets

This method enables the generation of datasets with varying numbers of attributes and records, capturing different levels of dependency strength and noise in attribute relationships.

The statistics of the datasets are presented in Table 2.

Algorithms Implemented. We implement and demonstrate results on six different algorithms for the 3 error measures that are studied in the paper. The first 3 algorithms are based on the upper and lower bounds. AppxIPF is our machine-learning based baseline, whereas, Pyro++ and TANE are the non-ML based baselines.

- *ExLP*: exact algorithm using linear programming based lower and upper bound.
- *ExDCS*: exact algorithm which uses a relaxed version of *ExLP*.
- *AppxLP*: approximate linear programming based uninformed prior algorithm.
- *AppxIPF*: Iterative proportional fitting (IPF) is an ML based approximate algorithm that uses informed prior and adapts the RAS algorithm [40]. Informed prior assumes that the future data-distribution follows a similar pattern as the read data-distribution while the marginal constraints are satisfied. The approach iteratively adjusts the weights of the initial probability distribution to satisfy marginal constraints using maximum-entropy-like modeling. We use the `python` library function in the `ipfn` package.
- *Pyro++* [23]: SOTA solution that uses sampling to identify UCCs (unique column combinations) that are likely to satisfy an error threshold. Then, it uses a BFS based lattice traversal to find minimal and non-trivial AFDs. Since it uses sampling, it can miss out some AFDs - hence this is not an exact solution.
- *TANE* [18]: The original implementation produces all AFDs within an error threshold ϵ. We take the original solution and adaptively update ϵ such that only k AFDs are produced as output.

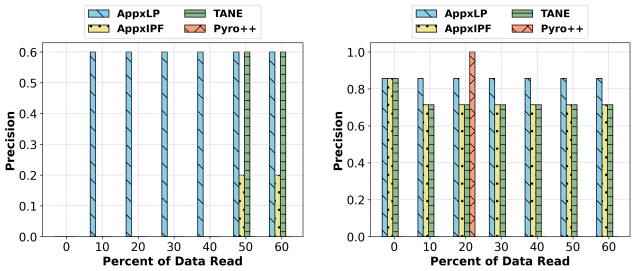
Hardware and Software: Our experiments are run on a machine with Intel Core i7-6850K CPU@3.60GHz processor and 64GB amount of RAM. The system uses Ubuntu version 22.04.3. All algorithms are implemented in `python 3` . Our code and datasets used in the experiments can be accessed here .

Measures: For quality, we present three measures, precision [9], Normalized Discounted Cumulative Gain (NDCG) [45], and Recall based on the actual top-k AFDs. Runtime is measured in seconds.

5.2 Quality Analyses- Approximate Algorithms

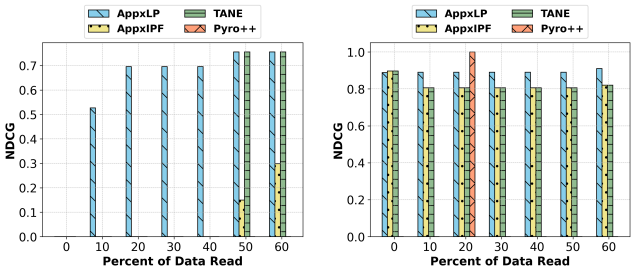
We present quality analyses of the approximate algorithm's results on top-k AFDs and ϵ-AFDs of length $l = 3$, considering different data organization strategies. We vary percentage of data read and measure precision and NDCG varying k. When *Pyro++* is used for comparison, we use g_1 as the error measure. The default error measure is otherwise g_3.

5.2.1 Data Indexed by Key Like Attributes. Here we organize data indexed by key like attributes, and measure effectiveness of the different algorithms. Figures 2a and 3a show precision and NDCG of the DB Status dataset for $k = 5$. *AppxLP* is easily able to outperform



(a) Precision at $k = 5$, DB Status **(b) Precision at $k = 7$ for Letter**

Figure 2: Precision for top-k AFDs with $l = 3$ when data is indexed by key like attributes



(a) NDCG at $k = 5$ for DB Status **(b) NDCG at $k = 7$ for Letter**

Figure 3: NDCG for top-k AFDs with $l = 3$ when data is indexed by key like attributes

all baselines. *TANE* is able to perform better after reading about half of the dataset. However, *Pyro++* performs poorly, since it is not exact and hence misses out some top-k AFDs. Thus the setting aids in revealing clear dependency patterns early in the top-k AFD exploration, helping the algorithm to perform better. However, this setting might not aid informed prior approach to perform better and the algorithm might overfit to expected patterns instead of real patterns, resulting in a comparatively less efficient performance. Figures 2b and 3b show these results for the Letter dataset for $k = 7$. These results demonstrate that *AppxLP* is consistently able to maintain a precision of approximately 0.85. *AppxIPF* and *TANE* also maintain a comparatively lower precision of approximately 0.7 throughout. *Pyro++* is only able to detect top-k AFDs at 20%.

5.2.2 Data Indexed by Non Key Like Attributes. Here we organize data indexed by non-key like attributes, and measure effectiveness of the different algorithms. Figures 4a and 4c show these results for the Letter dataset for $k = 7$. *AppxLP* achieves a precision score of nearly 0.85 after reading only a single record. *AppxIPF* and *TANE* initially perform as good as *AppxLP* but eventually drop to approximately 0.7 precision at 10%. *Pyro++* has the least precision throughout. Figures 4b and 4d show these results for the Student dataset for $k = 3$. *AppxLP* again outperforms all baselines after reading only 10% of the data and achieving a precision and NDCG score of 1. All other algorithms have a high precision except for *Pyro++*, which does not perform as well in this setting. However, uninformed prior approach *AppxLP* did the best job because of its ability to explore patterns more freely in a disordered setting. These patterns might be missed by the informed prior approach as it favors expected dependencies.

Table 3 reports the recall scores for the total number of ϵ-AFDs identified by our approximate methods at $e = 0.01$ across seven datasets. *AppxIPF* achieved a perfect recall of 1.0 on five out of

the seven datasets, while *AppxLP* attained high recall scores on majority of the datasets.

5.2.3 ϵ-AFDs. We evaluate our approximate algorithms to find all minimal AFDs with an error value below a given threshold, focusing on precision rather than ranked output. For the DB Status dataset, with a 0.01 error threshold, *AppxLP* achieves a precision of 1.0 after reading only 10% of the data, while *AppxIPF* reaches it after 20% (Figure 5a). *TANE* maintains a precision of 0.6, and *Pyro++* fails to generate any valid AFDs. When the dataset is sorted by key-like attributes (Figure 5b), *AppxLP* maintains precision of 1.0 after reading only 10% of data, *AppxIPF* reaches precision of 1.0 after 50%, while *TANE* and *Pyro++* show lower and zero precision respectively. Figure 5c shows the results for the DB Status dataset sorted by non-key like attributes, where similar observation holds. These results demonstrate the effectiveness *AppxLP* throughout.

5.3 Quality Analyses - Exact Algorithms

We evaluate our exact solution using the g_3 error measure to find top-k AFDs of length $\ell = 1$ on two synthetic datasets with controllable data distributions. Results show that after reading 40% and 30% of the data, our solutions accurately identify the top-k AFDs for $k = 5$ and $k = 7$ (Figures 7a and 7b respectively).

Figures 8a and 8b show comparison of our Exact algorithms with approximate algorithms and baselines. *AppxIPF* and *TANE* have a precision of approximately 0.3 until 30% data is read. *ExLP* and *ExDCS* are able to outperform approximate approaches after reading 40% data with a precision of 1.0. *Pyro++* achieves a precision of 1.0 at 50% data read.

5.4 Scalability Analyses

We measure the time for computing bounds and algorithm termination in both exact and approximate algorithms. For exact, we use one of our synthetic datasets whereas for approximate, we use DB Status dataset. Results show that *ExLP* is approximately 39.75% slower than *ExDCS*. For exact algorithms, *ExLP* is 102.37% slower than *ExDCS* for $k = 3$, significantly reducing to 13.14% for $k = 5$ and 3.73% for $k = 7$. For approximate algorithms, *AppxLP* is approximately 8.05% faster than *AppxIPF* on average. The results indicate that *AppxLP* is around 7.49% faster than *AppxIPF* for $k = 3$, 5.75% for $k = 5$ and 10.74% for $k = 7$. Figure 6 contains these results.

Table 4 summarizes an ablation study assessing the runtime of various components in the framework. For g_3, lower bounds are estimated using Mixed LP and DCS, and upper bounds via Exact LP. For g_1, QP is used for the upper bound, and QP with Ignore Row Sum for the lower bound. Depending on the strategy, the framework then computes either exact or approximate solutions. Runtime tests on a dataset of 10,000 rows demonstrate that DCS is the fastest for lower bounds, followed by Mixed LP and QP. For upper bounds, Exact LP is quicker than QP.

5.5 Summary

- Our designed algorithms exhibit anytime property. We observe that our designed solutions are highly accurate most of the time after reading 40% of the data. As expected, the effectiveness of the approximate algorithms are not always monotonic, but, in the long run (as we read more data), the effectiveness improves considerably. Contrarily, the exact algorithm demonstrates monotonic improvement.

Dataset	# Records	# Attributes	Recall AppxLP	Recall AppxIPF
Student	1000	5	0.7142	1.0
Letter	20,000	5	0.7142	1.0
DB Status	10,000	7	0.7307	0.7307
Synthetic 1	100,000	10	0.9583	1.0
Synthetic 2	100,000	10	0.9504	1.0
Synthetic 3	160,000	8	0.9310	1.0
Synthetic 4	48,000	16	0.9737	0.9887

Table 3: Recall for $e = 0.01$ after reading 60% data

Algorithm	Bound Type	Error	Runtime (sec)
Exact LP	Upper Bound	g_3	$5.15 * 10^{-4}$
Mixed LP	Lower Bound	g_3	$6.62 * 10^{-4}$
DCS	Lower Bound	g_3	$9.8 * 10^{-6}$
QP	Upper Bound	g_1	$1.19 * 10^{-2}$
QP, Ignore Row Sum	Lower Bound	g_1	$2.6 * 10^{-3}$

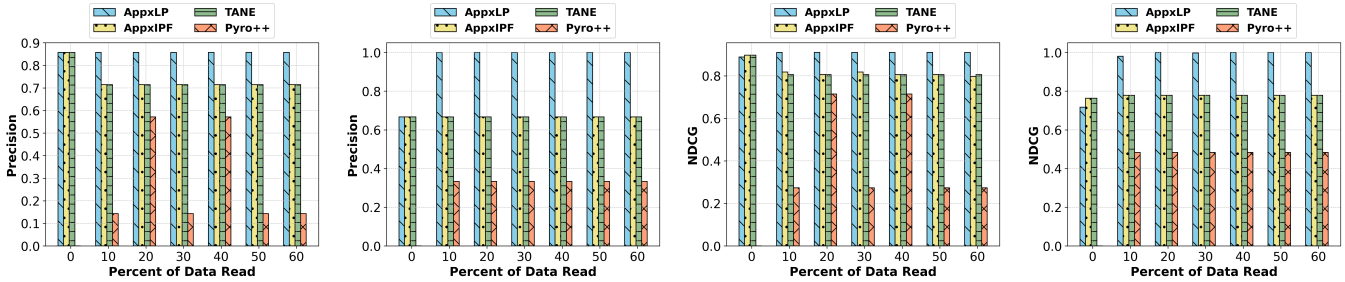
Table 4: Ablation Study

- The uninformed prior based approximate algorithm tends to outperform all three baselines by a decent margin for different datasets with different data indexing.
- Applications that prioritize faster processing times but can tolerate some level of inaccuracy, should opt for approximate algorithms that use informed priors. On the other hand, applications that require high accuracy but can afford longer processing times, benefit from using exact algorithms based on uninformed priors.

6 Related Work

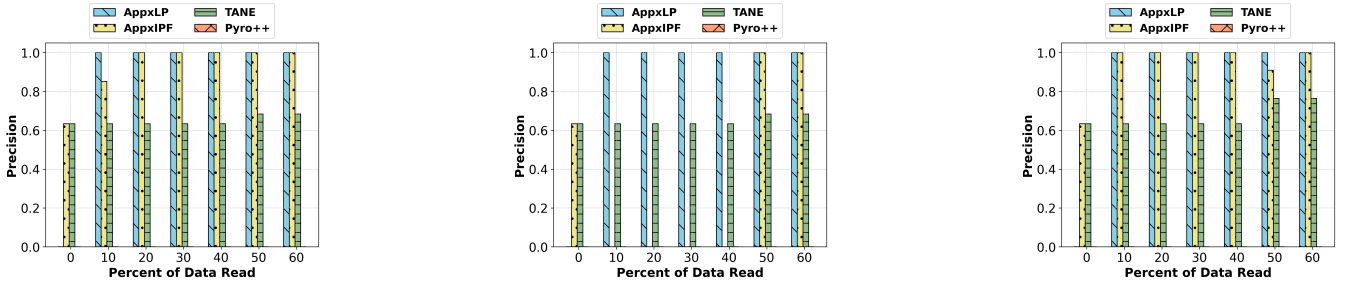
AFD and Applications. Functional Dependency (FD)[42] plays a crucial role in data preprocessing, cleaning, and integration[6, 31, 38]. Flach [14] approached FD discovery as an induction task aimed at concept definitions that classify examples accurately and generalize well. To handle challenges in FD discovery, Kivinen et al.[21] and Parciak et al. [34] study different error measures of AFDs. Most existing approaches rely on full database access and focus on g_1 or g_3 error measures, often using sampling for efficiency. Kolahi et al.[22] defined optimal repairs as databases that satisfy all FDs while minimizing a correction-based distance (a variant of g_3), and proved that verifying such repairs is NP-complete. We revisit the three error measures proposed by Kivinen et al.[21] — g_1, g_2, and g_3 — noting that g_2 remains the least utilized. These measures find application in various domains, including malicious account detection[8], data cleaning [38], and discovering FDs in incomplete data [7]. Le et al.[26] established a connection between g_3 error and classification accuracy upper bounds. Moreover, g_1 has been adopted to assess approximate denial constraints in data cleaning[10, 27, 35–37].

Incremental and Online Algorithm. Incremental, anytime, and online algorithms are all designed to operate under constrained environment, but each targets a different type of limitation. Incremental algorithms[39] focus on efficiently updating their output as new data arrives, avoiding the need to recompute results from scratch. These algorithms are especially useful in dynamic or streaming environments where data evolves over time. Online algorithms[1], on the other hand, process inputs sequentially and make irrevocable decisions at each step without knowledge of future inputs. In contrast, we focus on a subclass of anytime algorithms that generates effective output even after reading only a small portion of the data, and progressively improve as more data is processed.



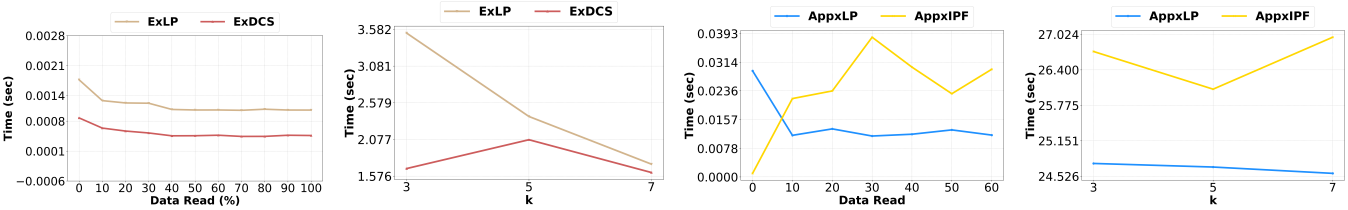
(a) Precision at $k = 7$ for Letter (b) Precision at $k = 3$ for Student (c) NDCG at $k = 7$ for Letter (d) NDCG at $k = 3$ for Student

Figure 4: Precision and NDCG for top-k AFDs with $l = 3$ when data indexed by non-key attributes



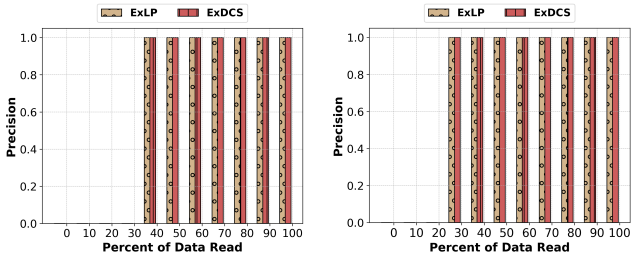
(a) Indexed arbitrarily (b) Indexed by key attributes (c) Indexed by non-Key attributes

Figure 5: Precision of $\epsilon = 0.01$ for ϵ-AFDs with $l = 3$ on DB Status



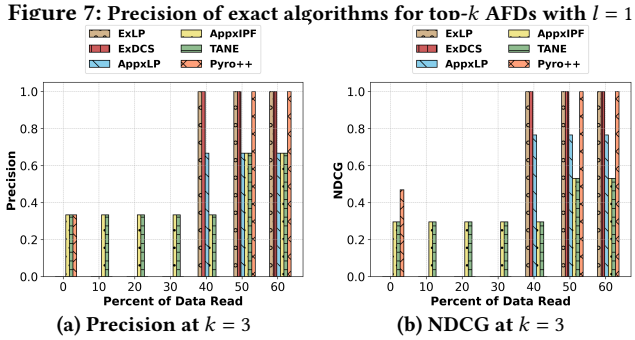
(a) Bound computation, *Exact* (b) Running time for Exact (c) Bound computation, *Approx* (d) Running time for Approx

Figure 6: Running time analysis



(a) $k = 5$ Synthetic Dataset 1 (b) $k = 7$ Synthetic Dataset 2

Figure 7: Precision of exact algorithms for top-k AFDs with $l = 1$



(a) Precision at $k = 3$ (b) NDCG at $k = 3$

Figure 8: Comparison of Exact, Approximate Algorithms with Baselines on Synthetic Dataset

7 Conclusion and Future Work

We present a computational framework designed to find AFDs *in anytime fashion* within a relation that generalizes to multiple error measures. We formalize two core sub-problems: maximum and minimum error estimates, which calculate the upper and lower bounds of an AFD's error based on data that is read . We systematically analyze the bound estimation problems and propose a suite of algorithms. We present exact and approximate lattice based algorithms for finding anytime AFDs using these bounds. We are the first to design solutions with anytime behavior. We conduct experiments to demonstrate the effectiveness of these algorithms under different data organization strategies and provide practical guidelines.

An interesting area of future work could consider partial/full marginals which are known *apriori*, but could have errors in them. Another interesting direction could be to design approximate, but faster approaches based on Dynamic Programming or other techniques to estimate the lower/upper bounds of g_1, g_2 and g_3 errors, which can be readily plugged into our framework.

Acknowledgements

The work of Senjuti Basu Roy was supported by : (1) NSF award number(s): 1942913, 2007935, 2) ONR award number(s): N000141812838, N000142112966, N000142412466. The work of Gautam Das was partly supported by NSF grants 2107296 and 2008602.

References

[1] Susanne Albers. 2003. Online algorithms: a survey. *Mathematical Programming* 97 (2003), 3–26.

[2] Benjamin Arai, Gautam Das, Dimitrios Gunopulos, and Nick Koudas. 2009. Anytime measures for top-k algorithms on exact and fuzzy data sets. *The VLDB Journal* 18 (2009), 407–427.

[3] Abolfazl Asudeh, Saravanan Thirumuruganathan, Nan Zhang, and Gautam Das. 2016. Discovering the skyline of web databases. *Proc. VLDB Endow.* (2016).

[4] Richard Bellman. 1966. Dynamic programming. *science* 153, 3731 (1966), 34–37.

[5] George Beskales, Ihab F Ilyas, and Lukasz Golab. 2010. Sampling the repairs of functional dependency violations under hard constraints. *Proc. VLDB Endow.* (2010).

[6] Tobias Bleifuß, Thorsten Papenbrock, Thomas Bläsius, Martin Schirneck, and Felix Naumann. 2024. Discovering Functional Dependencies through Hitting Set Enumeration. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–24.

[7] Bernardo Breve, Loredana Caruccio, Vincenzo Deufemia, Giuseppe Polese, et al. 2022. RENUVER: A Missing Value Imputation Algorithm based on Relaxed Functional Dependencies.. In *EDBT*. 1–52.

[8] Loredana Caruccio, Gaetano Cimino, Stefano Cirillo, Domenico Desiato, Giuseppe Polese, and Genoveffa Tortora. 2023. Malicious Account Identification in Social Network Platforms. *ACM Transactions on Internet Technology* 23, 4 (2023), 1–25.

[9] Stefano Ceri, Alessandro Bozzon, Marco Brambilla, Emanuele Della Valle, Piero Fraternali, Silvia Quarteroni, Stefano Ceri, Alessandro Bozzon, Marco Brambilla, Emanuele Della Valle, et al. 2013. An introduction to information retrieval. *Web information retrieval* (2013), 3–11.

[10] Xu Chu, Ihab F Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *Proceedings of the VLDB Endowment* 6, 13 (2013), 1498–1509.

[11] Arjun Dasgupta, Xin Jin, Bradley Jewell, Nan Zhang, and Gautam Das. 2010. Unbiased estimation of size and other aggregates over hidden web databases. In *ACM SIGMOD*. 855–866.

[12] Mateus de Oliveira Oliveira. 2010. Hasse diagram generators and Petri nets. *Fundamenta Informaticae* 105, 3 (2010), 263–289.

[13] Mark De Reuver, Hosea Ofe, Wirawan Agahari, Antragama Ewa Abbas, and Anneke Zuiderwijk. 2022. The openness of data platforms: a research agenda. In *Proceedings of the 1st International Workshop on Data Economy*. 34–41.

[14] Peter A Flach and Iztok Savnik. 1999. Database dependency discovery: a machine learning approach. *AI communications* 12, 3 (1999), 139–160.

[15] Michael R Garey and David S Johnson. 2002. *Computers and intractability*. Vol. 29. wh freeman New York.

[16] Cunjing Ge. 2024. Approximate Integer Solution Counts over Linear Arithmetic Constraints. In *AAAI*, Vol. 38. 8022–8029.

[17] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. https://www.gurobi.com

[18] Yka Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1999. TANE: An efficient algorithm for discovering functional and approximate dependencies. *The computer journal* 42, 2 (1999), 100–111.

[19] Kaggle. 2024. Kaggle. https://www.kaggle.com/datasets/muhammadroshaanriaz/students-performance-dataset-cleaned Accessed: 2024-11-29.

[20] Ravi Kannan and Santosh Vempala. 1997. Sampling lattice points. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*. 696–700.

[21] Jyrki Kivinen and Heikki Mannila. 1995. Approximate inference of functional dependencies from relations. *Theoretical Computer Science* 149, 1 (1995), 129–149.

[22] Solmaz Kolahi and Laks VS Lakshmanan. 2009. On approximating optimum repairs for functional dependency violations. In *ICDT*. 53–62.

[23] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *Proceedings of the VLDB Endowment* 11, 7 (2018), 759–772.

[24] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *Proc. VLDB Endow*, (2018).

[25] Tuong Le and Bay Vo. 2016. The lattice-based approaches for mining association rules: a review. *WIREs DMKD* 6, 4 (2016), 140–151.

[26] Marie Le Guilly, Jean-Marc Petit, and Vasile-Marian Scuturici. 2020. Evaluating classification feasibility using functional dependencies. *Transactions on Large-Scale Data-and Knowledge-Centered Systems XLIV: Special Issue on Data Management–Principles, Technologies, and Applications* (2020), 132–159.

[27] Ester Livshits, Alireza Heidari, Ihab F Ilyas, and Benny Kimelfeld. 2020. Approximate denial constraints. *arXiv preprint arXiv:2005.08540* (2020).

[28] Panagiotis Mandros, Mario Boley, and Jilles Vreeken. 2017. Discovering reliable approximate functional dependencies. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 355–363.

[29] Renée J Miller, Mauricio A Hernández, Laura M Haas, Lingling Yan, CT Howard Ho, Ronald Fagin, and Lucian Popa. 2001. The Clio project: managing heterogeneity. *ACM Sigmod Record* 30, 1 (2001), 78–83.

[30] Ullas Nambiar and Subbarao Kambhampati. 2004. Mining approximate functional dependencies and concept similarities to answer imprecise queries. In *WebDB 2004*.

[31] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional dependency discovery: An experimental evaluation of seven algorithms. *Proceedings of the VLDB Endowment* 8, 10 (2015), 1082–1093.

[32] Thorsten Papenbrock and Felix Naumann. 2016. A hybrid approach to functional dependency discovery. In *SIGMOD*.

[33] Thorsten Papenbrock and Felix Naumann. 2017. Data-driven Schema Normalization.. In *EDBT*, Vol. 17. 342–353.

[34] Marcel Parciak, Sebastiaan Weytjens, Niel Hens, Frank Neven, Liesbet M Peeters, and Stijn Vansummeren. 2024. Measuring Approximate Functional Dependencies: a Comparative Study. In *ICDE*.

[35] Eduardo HM Pena and Eduardo Cunha de Almeida. 2018. BFASTDC: A bitwise algorithm for mining denial constraints. In *DEXA 2018*. Springer, 53–68.

[36] Eduardo HM Pena, Eduardo C De Almeida, and Felix Naumann. 2019. Discovery of approximate (and exact) denial constraints. *Proc. VLDB Endow.* (2019).

[37] Eduardo HM Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *Proceedings of the VLDB Endowment* (2022).

[38] Abdulhakim Qahtan, Nan Tang, Mourad Ouzzani, Yang Cao, and Michael Stonebraker. 2020. Pattern functional dependencies for data cleaning. *Proceedings of the VLDB Endowment (PVLDB)* 13, 5 (2020), 684–697.

[39] Ganesan Ramalingam and Thomas Reps. 1996. An incremental algorithm for a generalization of the shortest-path problem. *Journal of Algorithms* (1996).

[40] Ludger Ruschendorf. 1995. Convergence of the iterative proportional fitting procedure. *The Annals of Statistics* (1995), 1160–1174.

[41] Alexander Schrijver. 1998. *Theory of linear and integer programming*. John Wiley & Sons.

[42] Avi Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts, Seventh Edition*. McGraw-Hill Book Company.

[43] Yasin N Silva and Spencer Pearson. 2012. Exploiting database similarity joins for metric spaces. *Proceedings of the VLDB Endowment* 5, 12 (2012), 1922–1925.

[44] Juan Pablo Vielma. 2015. Mixed integer linear programming formulation techniques. *Siam Review* 57, 1 (2015), 3–57.

[45] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, and Tie-Yan Liu. 2013. A theoretical analysis of NDCG type ranking measures. In *Conference on learning theory*. PMLR.

[46] Nan Zhang and Gautam Das. 2011. Exploration of deep web repositories. *Proceedings of the VLDB Endowment* 4, 12 (2011), 1506–1507.

[47] Shlomo Zilberstein. 1996. Using anytime algorithms in intelligent systems. *AI magazine* 17, 3 (1996), 73–73.

A Intractability Results under Certain Assumptions

We consider a simplified setting with two attributes, X and Z, and availability of their marginal distributions without the database as inputs. Using this limited information, we analyze the computational complexity of determining whether the AFD $X \rightarrow Z$ can have a lower bound error of zero ($\epsilon = 0$).

Theorem A.1. *Consider two attributes X, Z with domain sizes m_X and m_Z given as inputs. Additionally, their frequency distributions $\mathcal{M}^X_{\mathcal{A}}$ and $\mathcal{M}^Z_{\mathcal{A}}$ are provided explicitly. The decision problem:* Does there exist a database such that the given marginals $\mathcal{M}^X_{\mathcal{A}}$ and $\mathcal{M}^Z_{\mathcal{A}}$ are satisfied, while the g_3 (or g_1, g_2) error of the AFD $X \rightarrow Z$ is zero *- is $\mathcal{NP}$-hard.*

Proof. We prove the $\mathcal{NP}$ harness by a reduction from Subset Sum [15]. For this reduction, we transform Subset Sum to the decision instance of g_3 error when 0 records are read.

The Subset Sum problem consists of a set of numbers $\mathcal{S}$ and a target T. In our instance of the g_3 problem, let the domain m_X of X be set to $|S|$ and the domain of Z set to 2 (i.e. $m_Z = 2$). The marginals for X, $\mathcal{M}^X_{\mathcal{A}}$ be set to S, i.e. the first element of S is the marginal for the first category of X and so forth. For Z, let the marginals, $\mathcal{M}^Z_{\mathcal{A}}$ be set to T, and $(\sum_{i=1}^{|S|} S[i]) - T$ respectively. The target for our g_3 error problem is set to 0. The idea behind this process is that if there exists a subset in S that adds up to T, there must exist an unseen joint distribution frequency matrix which has an error of 0,

i.e. there must exist an unseen joint distribution frequency matrix which has *no violations*.

The proofs for error measures g_1 and g_2 are similar to the one presented above, with the only difference being the change in error-functions (g_2 and g_3). □

The above proof relies solely on standard computational complexity theory. However, if we treat the database itself as part of the input, the aforementioned reduction from Subset Sum no longer applies, since the frequency values are bounded by the database size N. In this case, Subset Sum can be solved in polynomial time using dynamic programming.

B Proofs of theorems

Proof of Theorem 3.1:

Let $U^* \in \mathbb{R}^{m_X \times m_Z}$ be the optimal solution to the LP with error e^*. Let $U' = \lfloor U^* \rfloor$, be obtained by taking floor of each entry in U^*. While U' may violate the marginal constraints by having lower row/column sums, it never exceeds them.

We can add at most 1 to selected entries of U' so that each row and column sum matches the required marginals. Since there are m_X rows, at most m_X such adjustments are needed. Each adjustment may increase the error (e.g., the max term) by at most 1.

Hence, there exists an integer matrix with error at most $e^* + O(m_X)$, implying that the optimal integer solution has error at least $e^* - O(m_X)$. This proves the existence of an integer solution within additive error $O(m_X)$.

□

Proof of Theorem 4.1:

For any AFD $X \rightarrow Z$, adding attributes to the LHS can only reduce error; thus, ascendants have lower or equal error. To find minimal ϵ-AFDs, we start from single-attribute AFDs. If one satisfies the threshold, it is minimal; otherwise, Algorithm EXEA explore the lattice upward only if minimality is still possible. If an AFD cannot be resolved due to limited data, we defer exploration and wait for more input via `getNext()`. This ensures the current set $\mathcal{T} \subseteq OPT^*$, maintaining precision 1. As more data arrives, $\mathcal{T}$ grows monotonically, and if all AFDs are resolved, $\mathcal{T} = OPT^*$ with precision and recall both 1. The same reasoning applies to ESCA, with top-down lattice traversal. □

C Optimization Techniques

C.1 Bound Computation for g_1 Error

The error measure for g_1 is different compared to the other two error measures, as the computation of errors is based on violations between pairs of disagreeing data points.

Each record with $X = r$ and $Z = c$ is in violation with every other record with $X = r$ and $Z \neq c$. This could be represented as a sum of products, as below,

$$\sum_{r=0}^{m_X-1} \sum_{y=0}^{m_Z-2} \sum_{z=y+1}^{m_Z-1} (U[r][y]+S[r][y]) \times (U[r][z]+S[r][z])$$

One approach would be to use the above summation as on optimization problem with the linear marginal constraints defined in Section 3.3.1. However, for $|X| = 1$, further efficiency opportunity exists as we describe below.

$$\begin{aligned}
&minimize \ \sum_{r=0}^{m_X-1} \sum_{c=0}^{m_Z-1} (U[r][c] + S[r][c])^2 \\
&s.t. \ \forall r \in \{0, .., m_X - 1\} \ \forall c \in \{0, .., m_Z - 1\} \quad U[r][c] \geq 0 \\
&\qquad \forall r \in \{0, .., m_X - 1\} \quad row[r] = \sum_{a=0}^{m_Z-1} S[r][a] + U[r][a] \\
&\qquad \forall c \in \{0, .., m_Z - 1\} \quad col[c] = \sum_{a=0}^{m_X-1} S[a][c] + U[a][c]
\end{aligned}$$

Figure 9: Convex QP for UB of g_1 error

$$\begin{aligned}
&maximize \ \sum_{r=0}^{n-1} max[r] \\
&s.t. \ \forall r \in \{0, .., m_X - 1\} \ \forall c \in \{0, .., m_Z - 1\} \quad U[r][c] \geq 0 \\
&\qquad \forall r \in \{0, .., m_X - 1\} \quad row[r] = \sum_{a=0}^{m_Z-1} S[r][a] + U[r][a] \\
&\qquad \forall c \in \{0, .., m_Z - 1\} \quad col[c] = \sum_{a=0}^{m_X-1} S[a][c] + U[a][c] \\
&\qquad \forall r \in \{0, .., m_X - 1\} \quad \forall \, c \in \{0, .., m_Z - 1\} \\
&\qquad\quad max[r] \geq S[r][c] + U[r][c] \\
&\qquad\quad max[r] \leq S[r][c] + U[r][c] + (1 - \mathcal{B}[r][c]) * N \\
&\qquad\quad 0 \leq \mathcal{B}[r][c] \leq 1 \\
&\qquad \forall \, r \in \{0, \cdots, n-1\} \quad \sum_{c=0}^{m_Z-1} \mathcal{B}[r][c] = 1
\end{aligned}$$

Figure 10: LP relaxation for lower bound of g_3 error

C.1.1 Reformulate g_1 Error. Before we consider the reformulation, let us simplify the terms. For our reformulation, any sum with same indices for U and S will be simplified with, Mat i.e. $Mat[r][c] = U[r][c] + S[r][c]$. Based on this simplification, let us consider the below reformulation using known terms,

$$\sum_{y=0}^{m_Z-1} Mat[r][y] \times \sum_{z=0}^{m_Z-1} Mat[r][z] = \sum_{y=0}^{m_Z-1} \sum_{z=y+1}^{m_Z-1} Mat[r][y] \times Mat[r][z] + \sum_{y=0}^{m_Z-1} Mat[r][y] \times Mat[r][y]$$

But we know that $\sum_{y=0}^{m_Z-1} Mat[r][y] = row[r]$. Simplifying the above expression, we get,

$$(row[r])^2 = \sum_{y=0}^{m_Z-1} \sum_{z=y+1}^{m_Z-1} Mat[r][y] \times M[r][z] + \sum_{y=0}^{m_Z-1} (Mat[r][y])^2$$

Rearranging the terms we get,

$$\sum_{y=0}^{m_Z-1} \sum_{z=y+1}^{m_Z-1} Mat[r][y] \times Mat[r][z] = (row[r])^2 - \sum_{y=0}^{m_Z-1} (Mat[r][y])^2$$

Hence, our g_1 error function can be written as below,

$$\sum_{r=0}^{m_X-1} (row[r])^2 - \sum_{r=0}^{m_X-1} \sum_{y=0}^{m_Z-1} (Mat[r][y])^2$$

As $\sum_{r=0}^{m_X-1} (row[r])^2$ is a constant, our error function that we will use is as below,

$$- \sum_{r=0}^{m_X-1} \sum_{y=0}^{m_Z-1} (Mat[r][y])^2 \tag{1}$$

Based on lower or upper bounds, the above error function will be minimized or maximized. Note that as the above expression is a sum of squares with a negative sign, the function is concave. Hence, maximizing the error function can be done efficiently with any hill climbing technique.

C.1.2 Convex Quadratic Program. : Quadratic programming (QP) optimizes a quadratic objective under linear constraints. In the formulation, the constraints are derived from the marginal information $\mathcal{M}_{\mathcal{A}}^{X,Z}$, similar to those in the LP formulation (Section 3.3), while the objective function is based on Equation 1. By converting the maximization to a minimization via a sign change, we ensure convexity. The complete QP formulation is shown in Figure 9.

C.2 Lower Bound Computation Formulation

C.2.1 Further opportunities for Optimization. **Relaxing Partial Marginal Constraints**: We now study another potential relaxation based on "*dropping*" some marginal constraints. This is likely to make the solutions further efficient albeit more "loose". This technique utilizes the monotone nature of the error-measures to obtain lower bounds for the three error measures.

Let us consider the lower bound for g_3 error. As mentioned in Section 3.3, information about the marginals are enforced using linear constraints. For a given AFD $X \rightarrow Z$, the idea here is to drop some of these linear constraints on Z ("*right-hand side*") with the same optimization goal. This left us with an optimization sub-problem which can be efficiently and optimally solved greedily.

For the sub-problem for lower bound on g_3 error, the maximum value in each row needs to be maximized while the constraints on X are maintained. As the joint distribution frequency matrix captures the read data, the maximum value in the row represents the current maximum value in the read data. The rest of the values represent the current g_3 error. As we ignore the marginals on the columns of the final joint distribution frequency matrix, i.e. drop the constraints on Z, the current g_3 error is LB on the final error.

Algorithm 1: ESCA-Early Stop Criteria based Top-k AFD Algo

Input: Graph G with $\binom{d-1}{\ell}$ nodes
Output: Top-k' AFDs

$\mathcal{T} \leftarrow \emptyset$; update edges in G
for $i = 1$ **to** k **do**
  $best \leftarrow$ vertex in G with out-degree $|V| - 1$
  if *best exists* **then**
    Remove *best* from G; add its AFD to $\mathcal{T}$
    Add descendants of *best* to G and update edges
  else
    Remove nodes with in-degree $\geq k$, store G
    return $\mathcal{T}$
return $\mathcal{T}$

Algorithm 2: EXEA — Early Stop ϵ-AFD Algorithm

Input: Attribute lattice
Output: Minimal ϵ-AFDs $\mathcal{T}$

$\mathcal{T} \leftarrow \emptyset$; $C \leftarrow \{A_i \rightarrow Z \mid A_i \in \mathbb{A}\}$
while $C \neq \emptyset$ **do**
  for $FD \in C$ **do**
    lb, ub $\leftarrow$ Orc(FD)
    if $lb > \epsilon$ **then**
      remove FD
    else if $ub \leq \epsilon$ **then**
      $\mathcal{T} \leftarrow \mathcal{T} \cup \{FD\}$
  Add minimal ascendants of unresolved FDs to C
return $\mathcal{T}$

C.3 Heuristic Approximate AFD Algorithms

We present two efficient approaches based on the *Uninformed prior* assumption on the error distribution that work well in practice.

Mean heuristic: Section 3 explains several ways of computing the lower/upper bound of error for a given AFD. For this discussion, consider the tight values for lower/upper bounds. If the PDF of the error is given/computed, the expected value can be computed exactly. But, as we have seen before, computing the PDF is very expensive and is not practical. Instead of exact computation of the expected value, a quick but inexact estimate of the error can be computed by the mean of the (lower and upper) bounds, *i.e.* $\frac{(l+u)}{2}$.

To summarize, mean heuristic is a simple and fast heuristic to estimate the error scores based on the mean of the lower and upper bounds. The efficacy and efficiency of the heuristic is illustrated in the experiments section.

Sampling based approach A different heuristic we design is based on the assumption that the error is uniformly distributed between l and u, *i.e.* error$\in [l, u]$. Based on this assumption, the probability of the error value being equal to any integer in the range $[l, u]$ is $1/(u - l + 1)$ (normalized value).

We leverage this by sampling error scores uniformly from each AFD's range $[l, u]$ over all possible unread data distributions satisfying the marginals. These sampled scores will determine the top-k AFDs for a potential future distribution.

We repeat this process of sampling s times and record the ordered top-k AFDs for each sample. As each of these samples/runs are of uniform weight (we use unit weights), the set of top-k AFDs with the highest counts across s runs is the one that has the highest probability of being the actual-ordered top-k AFDs. We produce that ordered top-k as the output.

We analyze the running time of our Uninformed Prior Approach in the Lemma C.1 below.

Lemma C.1. *Our sampling based algorithm consumes $O(|\mathbb{S}|\ (s + \log(|\mathbb{S}|) + \alpha)$ time when s is the number of samples, α is the amount of total time needed to compute the lower and upper bounds.*

Proof. Let the number of samples used be s. In each iteration, we generate a top-k AFD by simulating the error generated based on the uninformed prior assumption. For the simulation, we need the lower/upper bounds for the $|\mathbb{S}|$ AFDs, each of which consumes α time. Additionally, finding the top-k AFDs after the individual errors are generated takes $O(|\mathbb{S}| \log(|\mathbb{S}|))$ time. Thus, the total time needed is $s\ |\mathbb{S}| + |\mathbb{S}| \log(|\mathbb{S}|) + \alpha|\mathbb{S}|$ for the $|\mathbb{S}|$. Thus, the overall complexity is $O(|\mathbb{S}|\ (s + \log(|\mathbb{S}|) + \alpha)$. □